



DEEP LEARNING

AULA 6



Prof.^a Me. Mariane Gavioli Bergamini



CONVERSA INICIAL

Atualmente, diferentes áreas do conhecimento (engenharias, ciências, biologia, negócios etc.) apresentam problemas e por isso se faz necessário encontrar como solução um conjunto de alternativas disponíveis sob determinadas restrições. Uma das principais maneiras de resolver esse tipo de problema é redigi-lo como um problema de otimização, transformando os objetivos em funções matemáticas, que podem obter valores máximos ou mínimos, bem como as variáveis que geram tais resultados.

Dessa maneira, a otimização pode ser considerada como a tarefa de se encontrar uma ou mais soluções que minimizam ou maximizam um ou mais objetivos que satisfaçam todas as restrições, quando houver. Um problema de otimização pode estar associado a algumas técnicas que são consideradas métodos eficientes para tratar de problemas de otimização. Portanto, toda vez que surgem novos problemas de otimização, os métodos heurísticos e meta-heurísticos são uma das principais ferramentas para tratar esses problemas reais.

O objetivo da nossa jornada é estudarmos alguns problemas importantes de otimização, como mono-objetivo e multiobjetivo. Vamos também explorar os diferentes métodos baseados em programação matemática, pesquisa operacional e heurísticas computacionais.

Vamos, ainda, explorar os métodos meta-heurísticos, que são capazes de apresentar resultados eficientes e agilidade na obtenção dos resultados. Além disso, outra seção é destinada ao estudo das meta-heurísticas híbridas. Esse método faz a junção de dois algoritmos meta-heurísticos, em que a melhor estratégia de busca por soluções é implementado em um segundo algoritmo meta-heurístico.

Esses métodos trazem soluções para os problemas de otimização, a fim de encontrar soluções satisfatórias. Os métodos meta-heurísticos foram projetados para solucionar problemas de grande dimensionalidade, pois são algoritmos de fácil implementação e robustos.

Por fim, vamos compreender o uso dos métodos de otimização para tratar dos problemas de otimização utilizando funções objetivo, ou *Benchmark Function*. Essas funções auxiliam na avaliação de desempenho, convergência e precisão do algoritmo.



TEMA 1 – PROBLEMAS DE OTIMIZAÇÃO

Ao longo dos tempos, os seres humanos sempre estiveram envolvidos com o processo de otimização. Desde sua forma primitiva, a otimização consistia em rituais e preconceitos não científicos (sacrificar animais aos deuses; consultar oráculos; observar as posições das estrelas, por exemplo). Quando as circunstâncias eram convenientes, o momento podia ser considerado como ótimo para tal situação, por exemplo: plantar algo ou bom momento para fazer guerra.

Os problemas de otimização na prática dependem de diversos parâmetros de um sistema, fatores de ruído, parâmetros incontroláveis etc. Esse sistema pode ser de energia elétrica, para fabricação de carros, uma máquina de raio x, parâmetros de materiais (tensões de escoamentos), coeficientes de atrito, momentos de inércia etc.

Quanto a esses parâmetros do sistema, é necessário fazer o seu modelo para sabermos quais são eles e quais são as suas funcionalidades. Posto isso, um modelo pode ser representado de maneira matemática, em que podemos ter total conhecimento do sistema (todos os parâmetros são conhecidos) ou apenas o conhecimento parcial (alguns parâmetros são conhecidos).

Com o avanço das tecnologias, os sistemas formulados como um problema de otimização vêm avançando e, com isso, uma dificuldade elevada para se obter bons resultados é apresentada. Na tentativa de compreendermos os sistemas olhando apenas a entrada e a saída, estes podem ser formulados como um problema de otimização mono-objetivo. Contudo, em razão do avanço dos sistemas, suas variáveis passaram a ser analisadas também para melhorar o comportamento do sistema e obter resultados precisos. Dessa maneira, o sistema poderia ser formulado como um problema de otimização multiobjetivo.

1.1 Otimização mono-objetivo

Muitos problemas concretos de economia, pesquisa operacional, ciências de dados, medicina etc. podem ser formulados por um problema de otimização mono-objetivo (do inglês *monobjective*). Sua fórmula pode ser dada como:

$$\max \text{ ou } \min f(x) \mid x \in X, X \subseteq S \quad (1.1)$$



Na equação 1.1, a função objetivo (meta) f pode ser maximizada ou minimizada. Ainda, S é o espaço solução que depende das restrições (limites inferior e superior), X é o conjunto factível (ou ótima) que pode ser representado por uma população de indivíduos, e $\mathbf{x}$ é uma única solução factível da suposta população de indivíduos. Vamos pensar que temos uma população de felinos, mais especificamente uma população de leões. Dentro da população de felinos, teremos um leão e três leoas e, para encontrar o *fitness* da população, vamos usar a seguinte equação matemática:

$$f(x) = -\frac{d(d+4)(d-1)}{6}, \text{ para } X_i = i(d+1-i) \text{ e } i = 1, 2, \dots, d. \quad (1.2)$$

Sendo $X_i \in [-d^2, d^2]$, a função vai avaliar dentro do intervalo $[1,10]$, que vai gerar aleatoriamente quatros valores para obter o valor de $\mathbf{x}$, que é a solução factível. A Figura 1 a seguir exemplifica o código em linguagem de programação.

Figura 1 – Exemplo de função objetivo mono-objetivo

```
# entrada X = [X1, X2, X3, X4]
import numpy as np

X = np.random.randint(1,10,4) # intervalo de 0 - 15
print(X)
d= len(X)
dd = [0, 1, 2, 3]
xx = X[0]
print(f'O tamanho da população é {d}')

xx1 = xx -1
sum1 = xx1^2
sum2 = 0

# print(f'O valor da somatória 1 é {sum1} e o valor da somatória 2 é {sum2}')

for i in dd:
    xi = X[i]
    xold = X[i-1]
    sum = xi-1
    sum1 = sum^2
    sum = xi*xold
    sum2 = sum + sum2
    y = sum1 - sum2
    print(f'O valor da somatória 1 é {sum1} e o valor da somatória 2 é {sum2}')

    print(f'O resultado da aptidão é {y}.')
```

Fonte: Bergamini, [S.d.].

Conforme a Figura 1 anterior, é possível analisarmos a aptidão dos quatro indivíduos da população de leões. Como se trata de um problema de otimização e minimização, então o melhor indivíduo dessa população será aquele com menor valor de aptidão, conforme a Figura 2 a seguir.



Figura 2 – Soluções dos indivíduos da população

```
[9 5 4 1]
O tamanho da população é 4
O valor da somatória 1 é 10 e o valor da somatória 2 é 9
O resultado da aptidão é 1.
O valor da somatória 1 é 6 e o valor da somatória 2 é 54
O resultado da aptidão é -48.
O valor da somatória 1 é 1 e o valor da somatória 2 é 74
O resultado da aptidão é -73.
O valor da somatória 1 é 2 e o valor da somatória 2 é 78
O resultado da aptidão é -76.
```

Fonte: Bergamini, [S.d.].

1.2 Otimização multiobjetivo

Na otimização multiobjetivo (do inglês *multiobjective optimization*), de modo geral, não existem soluções ótimas no sentido de minimizarem ou maximizarem individualmente todos os objetivos. Pode-se dizer que a principal característica dessa otimização é a existência de um conjunto de soluções eficientes, ou seja, um conjunto de soluções aceitáveis que são superiores às demais. Porém, encontrar esse conjunto de soluções aceitáveis é uma tarefa inviável quando se trata de problemas de otimização com muitos objetivos (várias funções objetivos). Por esse motivo, os desenvolvedores tentam entregar um conjunto com a maior quantidade de soluções factíveis e esse conjunto pode ser referenciado como conjunto de Pareto ou conjunto de Aproximação.

Um problema de otimização é uma situação que requer a escolha de um conjunto de alternativas possíveis, a fim de obter um benefício predefinido por maximização ou minimização. Problemas de otimização multiobjetivo podem ser formulados pela equação 1.2:

$$\min f(X) = [f_1(X), f_2(X), \dots, f_o(X)]$$

Sujeito a:

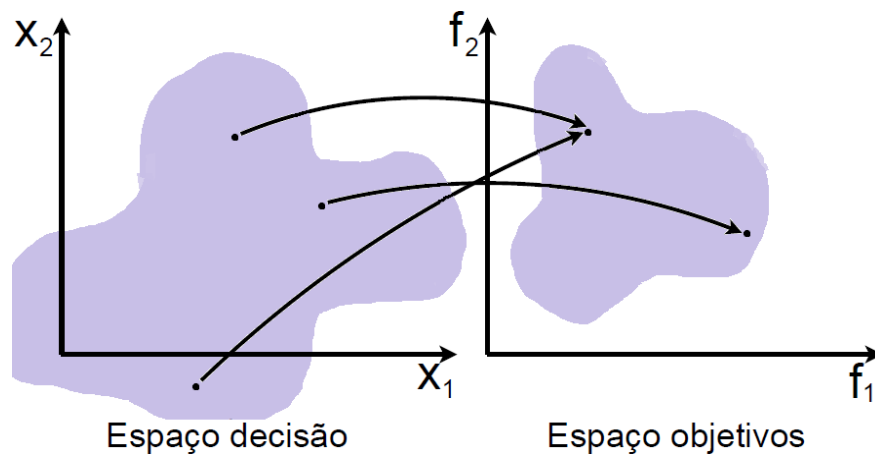
$$\begin{aligned} g_i(X_i) &\geq 0, & i &= 1, 2, \dots, m \\ h_i(X_i) &= 0, & i &= 1, 2, \dots, p \\ L_i &\leq X_i \leq U_i, & i &= 1, 2, \dots, n \end{aligned} \tag{1.3}$$

Na equação 1.3, **n** é o número de variáveis, **o** é o número de funções objetivo, **m** é o número de restrições de desigualdade, **p** é o número de restrições de igualdade, g_i é a i -ésima restrição de desigualdade, h_i indica a i -ésima

restrição de igualdade e $[L_i \text{ e } U_i]$ são os limites inferior e superior da i -ésima variável.

Além disso, na otimização multiobjetivo há dois espaços de busca a serem considerados: o primeiro é referenciado por espaço de decisão, que tem as variáveis de decisão; o segundo é o espaço objetivo, que contém os valores das funções objetivo. Na Figura 3 a seguir, podemos ver os espaços de busca bem como observar também que não é obrigatório que os dois espaços apresentem a mesma dimensão.

Figura 3 – Exemplo de mapeamento de pontos entre os espaços de decisão e objetivos



Fonte: Bergamini, [S.d].

A área azul da Figura 3 anterior representa o conjunto viável, em que o processo de busca de otimização ocorre dentro do espaço de decisão. Isso somente ocorre em decorrência da manipulação das variáveis, e uma análise de qualidade das soluções é realizada no espaço objetivo.

Uma solução para um problema de otimização multiobjetivo é considerada eficiente, ótima de Pareto ou não dominada se e somente se não houver outra solução viável para o problema, sendo que a solução X_1 domina a solução X_2 . Há o interesse em encontrar o maior número possível de soluções não dominadas, uma vez que elas apresentam os melhores *trade-offs* disponíveis entre os valores das funções objetivo.

Tabela 1 – Configurações da otimização multiobjetivo

Função objetivo	M-objetivo	Dimensão	Espaço de busca
DTLZ 1	3	$M + 9$	$[0,1]^D$

Fonte: Bergamini, [S.d].

A função DTLZ 1 (Deb, Thiele, Laumanns e Zitzler) é considerada como um problema de M-objetivo, visto apresentar uma frente de Pareto linear simples e uma certa dificuldade em questão à convergência da frente de Pareto. Essa dificuldade é por causa do espaço de busca poder levar os métodos de otimização a atingir antecipadamente a frente global de Pareto-ótimo, ou seja, uma convergência prematura. Na Figura 4 a seguir, é dado um exemplo de função multiobjetivo, em que a função teste é chamada de DTLZ.

Figura 4 – Código exemplo da função multiobjetivo DTLZ1

```
from pymoo.problems import get_problem
from pymoo.util.plotting import plot
from pymoo.util.ref_dirs import get_reference_directions
from pymoo.visualization.scatter import Scatter

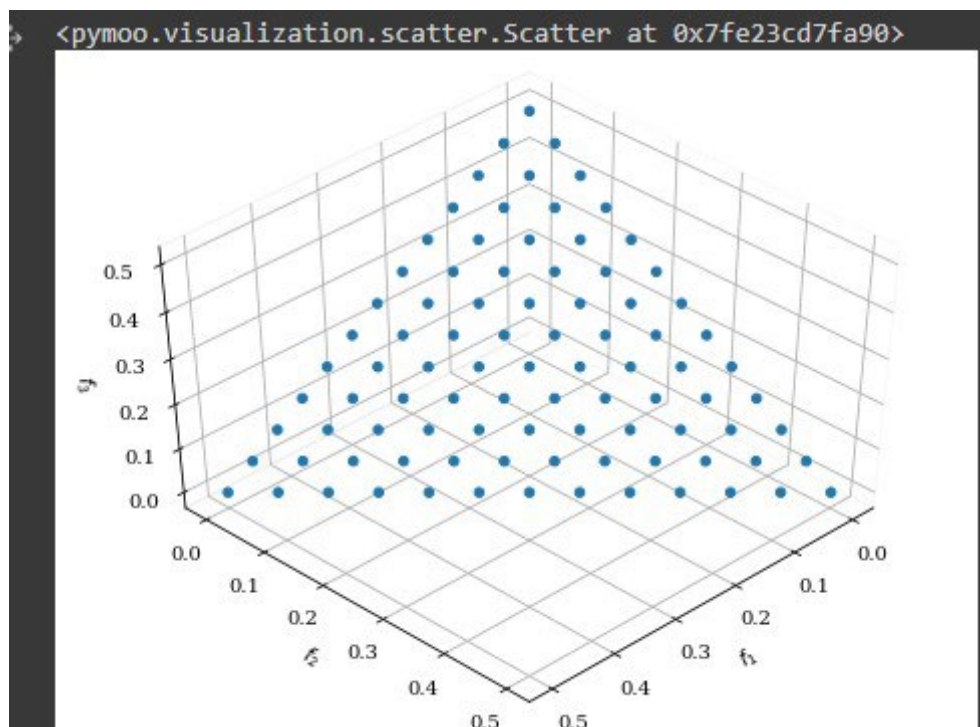
ref_dirs = get_reference_directions("das-dennis", 3, n_partitions=12)

pf = get_problem("dtlz1").pareto_front(ref_dirs)
Scatter(angle=(45,45)).add(pf).show()
```

Fonte: Blank, 2022. Multi-objective Optimization in Python.

Para uma compreensão gráfica, exemplificamos na Figura 5 a seguir soluções do conjunto de Pareto dentro do espaço de busca definido pela função DTLZ1.

Figura 5 – Conjunto de Pareto da função DTLZ1



Fonte: Blank, 2022. Multi-objective Optimization in Python.



O conjunto de Pareto corresponde a $x_i = 0,5$ ($x_i \in x_M$) e os valores da função objetivo estão dentro do hiperplano linear (olhando para a Figura 5 anterior, é possível visualizar o formato de um triângulo) definido por $\sum_{m=1}^M f_m^* = 0,5$, conforme mostrado na Figura 5 anterior.

Saiba mais

Para mais informações sobre problemas multiobjetivo na linguagem de programação Python, recomendamos acessar o *link* disposto a seguir. Disponível em: <<https://pymoo.org/>>. Acesso em: 19 out. 2022.

TEMA 2 – HEURÍSTICAS

Nos últimos anos, a otimização se tornou a área de pesquisa que fornece uma solução ideal para problemas de otimização complexos em tempo real. Mono-objetivo, multiobjetivo, diferenciabilidade e diferentes características combinatórias são enfrentadas com esses problemas. A procura por resolver problemas em tempo real tem chamado a atenção de vários cientistas para desenvolver algoritmos de otimização que sejam ágeis, precisos e computacionalmente poderosos.

Segundo Arroyo (2002), a escolha do método de resolução depende da razão entre a qualidade da solução gerada por determinado método e o tempo computacional que se gasta para encontrar a solução, que na maioria das vezes são ditas de problemas sem soluções, por conta de seu tempo computacional realizado pelo método heurístico.

Métodos heurísticos para otimização são desenvolvidos para tratar de maneira ágil problemas complexos de larga-escala, sendo impraticável o tempo computacional gasto pelos métodos ao encontrar um resultado. Dessa maneira, tais métodos podem usar o conhecimento parcial sobre o problema com o objetivo de pegar atalhos e encontrar boas soluções, que podem não ser o ótimo global. Com isso, as heurísticas conseguem manter o equilíbrio entre o esforço computacional e a qualidade da solução que se pretende obter. Para obter esse equilíbrio, é de suma importância que se tenha o conhecimento parcial sobre o problema, de modo a explorar mais adequadamente o espaço de busca. Geralmente, esse conhecimento pode orientar o algoritmo heurístico reduzindo o espaço de busca sob estudo. A seguir, estão algumas situações em que as heurísticas são requeridas:



1. oferecem mais de uma solução aceitável, possibilitando ampliar as chances de decisão, ou seja, fornecem o valor aproximado ao valor ótimo;
2. quando os métodos exatos prejudicam o tempo de execução, memória e tempo de desenvolvimento;
3. quando os dados não passam confiança, ou seja, a solução ótima não tem nada a ver com o problema;
4. quando a modelagem matemática não apresenta aspectos importantes do problema real; modelos matemáticos não conseguem ser representar problemas complexos. Com isso, as heurísticas são versáteis e de fácil implementação;
5. conseguem encontrar boas soluções inicialmente para os métodos exatos, como método do *branch-and-bound*, programação linear.

Um exemplo de heurística é a do vizinho mais próximo para o famoso problema do caixeiro viajante (do termo em inglês *Travelling Salesman Problem* - TSP).

2.2 Heurística construtiva: “vizinho mais próximo”

A heurística do “vizinho mais próximo” (do inglês *Nearest Neighbor Heuristic* – NNH) tenta construir ciclos hamiltonianos com base em conexões com os vizinhos próximos. De modo resumido, essa heurística funciona como:

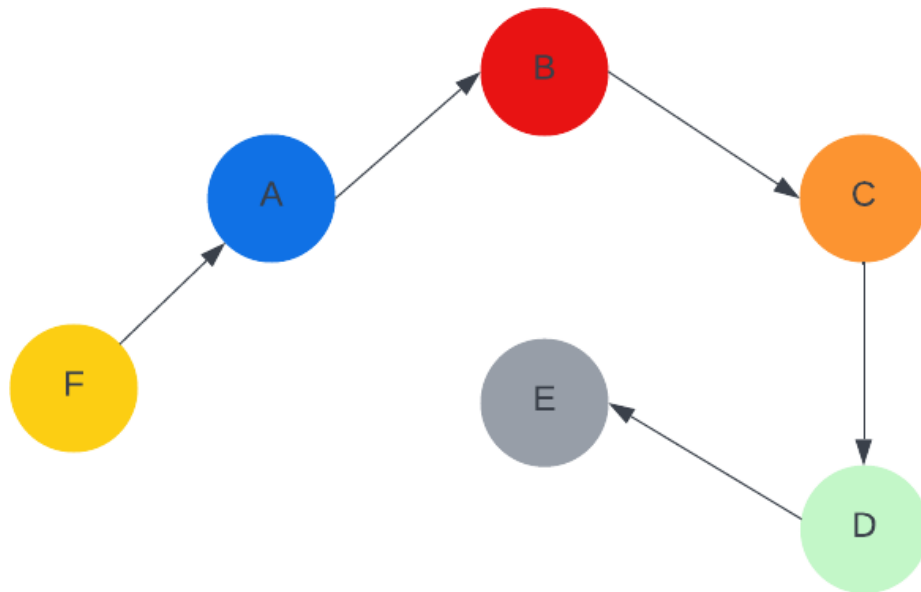
1. a heurística se inicializa arbitrariamente pelo vértice A;
2. passa por todos os vértices na tentativa de não passar pelo mesmo vértice; e
3. retorna ao vértice inicial.

Como uma tentativa de compreendermos melhor o problema do caixeiro viajante, será necessário resolvermos qual o caminho mais curto que podemos utilizar para visitar as cidades exatamente uma única vez e retornar à cidade de partida (inicial). Sendo um conjunto de cidades e a distância entre cada par de cidades dados, qual é o passeio mais curto possível que visita cada cidade exatamente uma vez e retorna à cidade inicial?

Na Figura 6 a seguir, demonstramos graficamente a ideia do problema do caixeiro viajante.



Figura 6 – Problema do caixeiro viajante



Fonte: Bergamini, [S.d.].

A principal ideia é obtermos a menor distância entre as cidades. Lembrando que é preciso passar por todas as cidades uma única vez, saindo da cidade F até a cidade E. Não importa a sua ordem, precisamos encontrar a melhor rota. Posto isso, ao aplicarmos a heurística construtiva NNH, teremos o seguinte código em Python, disposto na Figura 7 a seguir.

Figura 7 – Resolução do problema do caixeiro viajante

```
def nn_tsp(cities, start=None):
    """Inicie o passeio na cidade inicial fornecida (padrão: primeira cidade);
    a cada passo estende o passeio movendo-se da cidade anterior
    ao seu vizinho mais próximo que ainda não foi visitado."""
    C = start or first(cities)
    tour = [C]
    unvisited = set(cities - {C})
    while unvisited:
        C = nearest_neighbor(C, unvisited)
        tour.append(C)
        unvisited.remove(C)
    return tour

def first(collection): return next(iter(collection))

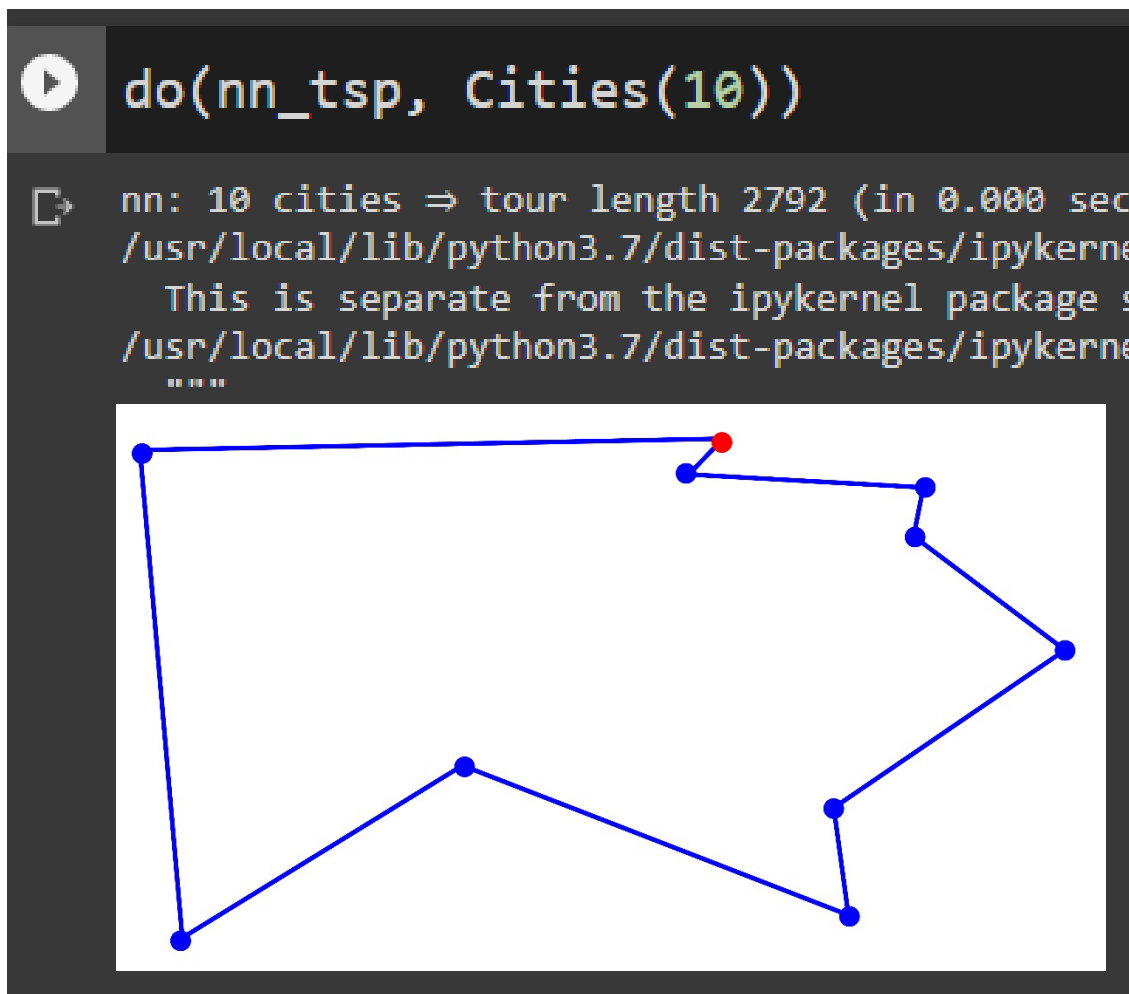
def nearest_neighbor(A, cities):
    "Find the city in cities that is nearest to city A."
    return min(cities, key=lambda C: distance(C, A))
```

Fonte: com base em Norvig, 2018.

De acordo com a Figura 7 anterior, para a variável “cities” foram criadas as coordenadas $x = 300$ e $y = 100$. Além disso, foi preciso pensar no cálculo para saber a distância entre duas cidades. A variável “unvisited” guarda as cidades que não foram visitadas e a variável C aloca o ponto de partida. Com isso, é possível analisar a distância entre as cidades por meio do ponto de partida.

Para uma última visualização dos resultados, na Figura 8 a seguir mostramos graficamente 10 cidades a serem percorridas.

Figura 8 – Percurso percorrido entre 10 cidades



Fonte: Norvig, 2018.

Saiba mais

Existem diferentes heurísticas que podem resolver o problema do caixeiro viajante. É possível conhecer algumas delas no *link* disposto a seguir. Disponível em: <<https://github.com/norvig/pytudes/blob/main/ipynb/TSP.ipynb>>. Acesso em: 19 out. 2022.



TEMA 3 – METAHEURÍSTICAS

Diversos autores, ao longo dos anos, desenvolveram diferentes meta-heurísticas conforme sua categoria, tais como: baseadas em população, na vida social de enxames e de animais, em fenômeno químico, em busca de vizinhança ou trajetória etc. Dessa maneira, as meta-heurísticas são métodos de otimização alternativos para resolver problemas dinâmicos de grandes escalas e complexos (sistemas de controle, turbina hidráulica, análise de dados estatísticos, construção de modelos etc.).

Na Figura 9 a seguir, podemos observar algumas meta-heurísticas que estão divididas de acordo com a categoria que caracterizam alguns processos, como biológico, físico, social etc. Além disso, as meta-heurísticas estão classificadas de acordo com sua categoria e têm sua própria estratégia para buscar soluções de maneira eficiente e simplificada (Mahdavi; Shiri; Rahnamayan, 2015).

Figura 9 – Exemplo de taxonomia das metaheurísticas

Evolução	Inteligência de enxames
Algoritmo genético	Partícula de enxame
Programação evolutiva	Colônia de formiga-leão
Evolução diferencial	Lobo cinzento
Fenômeno físico	Procedimentos humanos
Recozimento simulado	Busca tabu
Busca gravitacional	Busca guiada
Pesquisa de vórtice	Busca local iterativa

Fonte: Bergamini, [S.d.].

As meta-heurísticas, que tem por base a população, são algoritmos baseados na teoria de Darwin ou evolução. A teoria nada mais é que a evolução hereditária de indivíduos de uma mesma população, os quais passam pelos seguintes processos: cruzamento, mutação e seleção do melhor indivíduo. Algumas meta-heurísticas dessa categoria são: algoritmos evolucionários (do inglês *Evolutionary Algorithms* – EA); algoritmo genético (do inglês *Genetic Algorithm* – GA), evolução diferencial (do inglês *Differential Evolution* – DE), entre outros.

No caso da categoria “algoritmos evolutivos”, a evolução diferencial (do inglês *Differential Evolution* – DE) se destaca e razão da sua simplicidade para implementá-la e robustez ao buscar boas soluções. Essa meta-heurística



depende de dois operadores, operador de mutação e operador de cruzamento, que buscam por boas soluções combinando a estratégia de mutação com o operador de cruzamento em um conjunto de indivíduos (Price; Storn; Lampinen, 2005).

Como característica principal pode-se citar os operadores estocásticos, ou seja, são operadores que buscam aleatoriamente um conjunto de soluções que sejam aceitas de acordo com o critério de satisfação do problema a ser solucionado. Recentemente, vários métodos meta-heurísticos foram propostos com resultados interessantes. Tais abordagens usam como inspiração nossa compreensão científica de sistemas biológicos, naturais ou sociais, que em algum nível de abstração podem ser representados como processos de otimização. A escolha do método de resolução depende da aplicação entre a qualidade da solução gerada por tal método e o tempo que ela gasta para encontrar a solução. Na maioria das vezes, são problemas intratáveis, por conta do tempo de processamento executado pelo algoritmo (Arroyo, 2002).

Do ponto de vista computacional, Cuervas et al. (2018) cita quatro características computacionais que os métodos meta-heurísticos apresentam semelhantemente, como:

- uma ou mais populações de soluções candidatas são consideradas;
- essas populações mudam dinamicamente em decorrência da produção de novas soluções, ou seja, mudam por causa da aleatoriedade, uma característica das meta-heurísticas;
- uma função de condicionamento físico reflete a capacidade de uma solução para sobreviver e se reproduzir, seja ela de minimização ou maximização;
- vários operadores estocásticos são empregados para explorar uma exploração apropriada do espaço das soluções que depende de cada característica das meta-heurísticas.

3.1 Evolução diferencial

Em 1995, a evolução diferencial (ED) teve sua primeira publicação em forma de relatório técnico. Desde então, começou a ter uma ampla variedade de aplicações no mundo real (Price; Storn; Lampinen, 2005).



A evolução diferencial pode ser considerada uma meta-heurística baseada na população para resolver problemas de otimização no tempo contínuo. Por causa de sua simplicidade, eficácia e robustez, tornou-se gradualmente conhecida e aplicada em diversos campos. Seu desempenho está diretamente ligado à sua estratégia de geração do vetor experimental, o qual depende de dois fatores principais: os operadores de mutação (F) e de cruzamento (Cr). Características funcionais, tais como dimensão do problema (dim), número de mínimos locais e grau de dependência dos parâmetros, dão valores adequados aos parâmetros de otimização do algoritmo e a estratégia de geração do vetor de ensaio. Muitas análises teóricas e empíricas foram realizadas sobre os parâmetros de controle. Os valores relativos de F e Cr são definidos por valores fixos em um determinado intervalo, $[0,5 \ 1]$ para F e para Cr no intervalo $[0,7 \ 1]$, ou então definidos por valores médios por intermédio de estudos existentes que testaram esses valores, levando em conta a diversidade de convergência e utilizando informações de classificação dos indivíduos.

O pseudocódigo do algoritmo ED é apresentado no Quadro 1 a seguir (Bergamini, 2018).

Quadro 1 – Pseudocódigo do algoritmo ED

Pseudocódigo 1: evolução diferencial
Gerar população aleatoriamente
Avaliar a aptidão de cada indivíduo da população
Critério de parada
Mutação
Avaliar a aptidão de cada indivíduo da nova população
Cruzamento
Seleção
Avaliar a aptidão da nova população se critério de parada é válido fim se não voltar ao início

Fonte: Bergamini, [S.d.].

A seguir, será explanada cada linha do pseudocódigo do algoritmo ED apresentado.

3.1.1 População inicial

A população vetorial inicial $(x_{i,j})$ é composta por M indivíduos, escolhida aleatoriamente e deve cobrir o espaço de busca que compõe a solução. A população $(x_{i,j})$ é iniciada após definir os limites superior $(Bu_{i,j})$ e inferior $(Bl_{i,j})$ para cada indivíduo da população.

O vetor inicial dos indivíduos X_i é representado pela equação 3.1 a seguir:

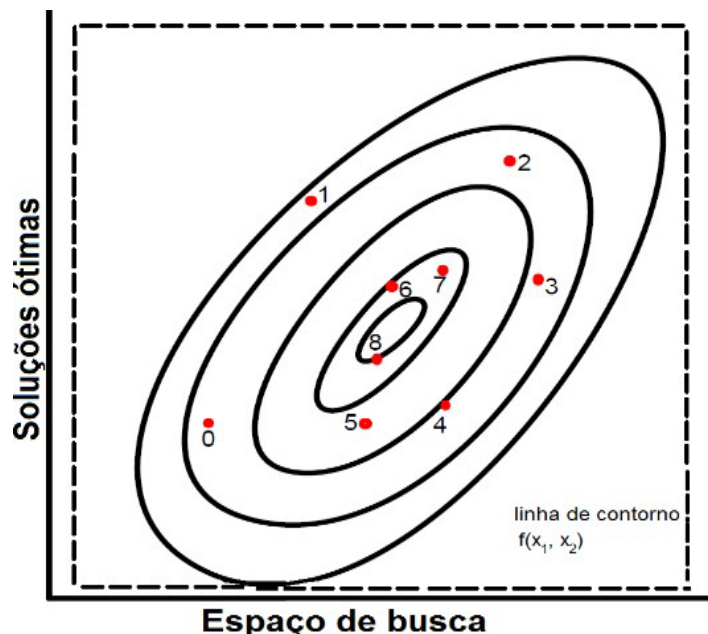
$$x_{i,j} = rand(M, dim)(Bu_{i,j} - Bl_{i,j}) + Bl_{i,j} \quad (3.1)$$

Nessa equação:

- M é o tamanho da população;
- dim é a dimensão do problema (número de variáveis estimadas);
- $i = 1, 2, \dots, M$;
- $j = 1, 2, \dots, dim$;
- $x_{i,j}$ é um indivíduo $\in X_i$.

Observando-se a Figura 10 a seguir, é possível compreender que a população de vetores é gerada dentro do espaço de busca (mín., máx.). Os indivíduos gerados aleatoriamente que estão perto do menor círculo correspondem aos ótimos globais, e, quanto mais longe estiverem do círculo, menor são ditos ótimos locais.

Figura 10 – Inicialização da população



Fonte: com base em Storn; Price, 1997.



A região de parâmetros é inteiramente coberta e representada pela linha de contorno $f(x_1, x_2)$. Assim, os vetores obtêm um índice de contabilidade para cada um deles ao entrarem na competição dentro do espaço de busca.

3.1.2 Função objetivo

Os indivíduos da população passam por uma avaliação de aptidão referenciada como função objetivo, função de custo ou função de aptidão (*fitness*). A função mono-objetiva trata da minimização ou da maximização do problema estudado, que pode ser dada por:

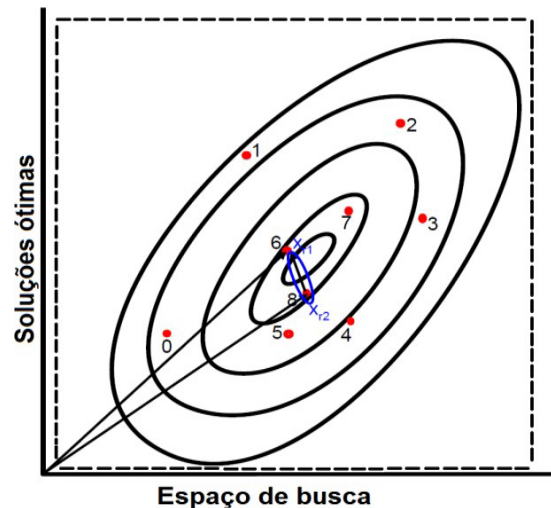
$$\min. f(x_i) = f(x_1, x_2, \dots, x_M) \quad (3.2)$$

A equação 3.2 representa a função de custo, pois se trata do erro de predição na função, em que é utilizada a minimização. Os valores dos indivíduos avaliados são $x_1, x_2, \dots, x_M$, a função de custo é representada por f e o tamanho da população por M .

3.1.3 Mutação

No caso de soluções preliminares, a população pode ser gerada por adição de desvios aleatórios normalmente distribuídos para uma solução nominal. Assim, a ED gera novos vetores adicionando a diferença ponderada entre dois vetores populacionais a um terceiro vetor. Chamamos essa operação de “mutação” (F). Dessa maneira, são selecionados aleatoriamente dois vetores diferença (x_{r1} e x_{r2}) em que $r1$ e $r2$ são inteiros escolhidos aleatoriamente entre $[1, M]$. Na Figura 11 a seguir, mostramos os dois vetores e o vetor diferença dentro do espaço de busca da população.

Figura 11 – Vetor diferença



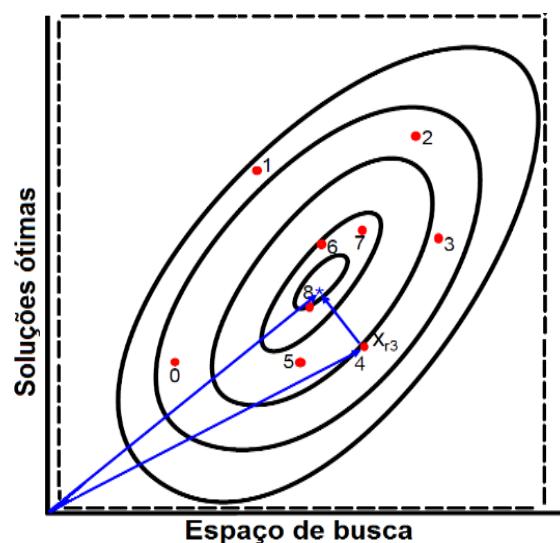
Fonte: com base em Storn; Price, 1997.

Os vetores diferença (x_{r1} e x_{r2}) são sinalizados pelas retas na Figura 11 anterior. Dessa maneira, ED sofre mutação e depois sofre recombinação para produzir uma nova população de vetores, ou seja, novos indivíduos são gerados. A mutação diferencial adiciona a diferença escalar ($x_{r1} - x_{r2}$) a um terceiro vetor x_{r3} aleatoriamente selecionado da população.

$$x_{r3} = rand(1, M) \quad (3.3)$$

O vetor x_{r3} ilustrado na Figura 12 a seguir é selecionado conforme a equação 3.3 e juntamente com o vetor de diferença ponderada produz o vetor experimental u_0 .

Figura 12 – Processo de mutação



Fonte: com base em Storn; Price, 1997.



Assim, o processo de mutação pode ser representado pela equação 3.4 a seguir, na qual será o resultado da diferença dos vetores x_{r1} e x_{r2} multiplicada pelo valor de mutação e somada com o vetor x_{r3} .

$$u_0 = x_{r3} + F(x_{r1} - x_{r2}) \quad (3.4)$$

Em vista de várias pesquisas realizadas, Zhang et al. (2015) apresentam um conjunto de múltiplas estratégias no operador mutação que são muito promissores, cujo desempenho pode ser melhorado por meio da combinação de várias estratégias de geração de vetor de teste com alguns parâmetros de controle adequados.

De acordo com os trabalhos de Ao e Chi (2009) e Kushida et al. (2015), existem diversas estratégias de mutação para a meta-heurística DE, as quais são formuladas como DE/x/y/z.

A letra **x** representa o tipo de vetor que sofrerá mutação; geralmente esse vetor é selecionado aleatoriamente dentro da população. Enquanto isso, **y** é a quantidade de vetor diferença que será utilizado na estratégia e **z** é o tipo de cruzamento a ser aplicado, podendo ele ser binominal (bin) ou exponencial (exp). A seguir, são citadas algumas estratégias utilizadas na meta-heurística DE.

- DE/best/1/exp
- DE/rand/1/exp
- DE/rand-to-best/1/exp
- DE/best/2/exp
- DE/best/1/bin
- DE/rand/1/bin
- DE/rand-to-best/1/bin
- DE/best/2/bin
- DE/rand/2/bin

Nesta etapa, é utilizada a estratégia *DE/best/1*, que pode ser formulada pela equação 3.5 a seguir

$$U_k = x_{best} + F(x_{r1} - x_{r2}) \quad (3.5)$$

Nessa equação:

- $x_{r1}, x_{r2} \in X_{i,j}$ são obtidos de forma aleatória;



- x_{best} é o valor de aptidão do melhor indivíduo da população;
- F é a taxa de mutação que será aplicada na população.

Outro detalhe nesse processo é que a taxa de cruzamento nesse caso é determinada de maneira aleatória, em outras palavras, gera um número aleatório por meio da distribuição uniforme contínua com o parâmetro inferior igual a 0 e o parâmetro superior igual a 1. Na equação 3.6, é descrito esse processo.

$$F = \text{unifrnd}(\text{beta_min}, \text{beta_max}, \text{VarSize}) \quad (3.6)$$

Nessa equação, beta_min é o limite inferior igual a 0,2, beta_max é o limite superior igual a 0,8, VarSize é um vetor de tamanho igual à dimensão do problema (dim) e unifrnd é um comando do Matlab® para gerar números aleatórios uniformes contínuos.

3.1.4 Cruzamento

Após o processo de mutação, ocorre o processo de cruzamento, chamado também de “recombinação discreta”. Os parâmetros do vetor mutado são misturados com os parâmetros de outro vetor pré-determinado, vetor alvo, responsável por produzir o vetor experimental. Essa mistura é referida como cruzamento (Cr). Sua probabilidade, $Cr \in [0 \ 1]$, é um valor definido pelo usuário que controla os valores dos parâmetros dos vetores mutantes copiados. Além disso, um número é gerado de forma aleatória uniforme, $\text{randj}(0,1)$. Se o número achado for menor ou igual a Cr , então o parâmetro experimental é herdado x_{ij} . O vetor atualizado u_{ij} é definido na equação 3.7 a seguir:

$$u_0 = \begin{cases} v_{ij} & \text{se } (\text{rand}(0,1) \leq Cr) \\ x_{ij} & \text{caso contrário} \end{cases} \quad (3.7)$$

Nessa equação, $\text{rand}(0,1)$ é um número gerado aleatoriamente no intervalo $[0 \ 1]$ e Cr é a taxa de cruzamento.

3.1.5 Seleção

O operador de seleção faz a comparação do vetor teste (u_0) com o vetor alvo (x_{ij}) para gerar os novos indivíduos. Se o valor de aptidão do vetor alvo for menor que o valor do vetor teste, o vetor alvo avança para a próxima geração,

caso contrário, o $x_{i,j}$ é substituído pelo valor de u_0 (Price; Storn; Lampinen, 2005). Esse processo pode ser descrito na equação 3.8 a seguir.

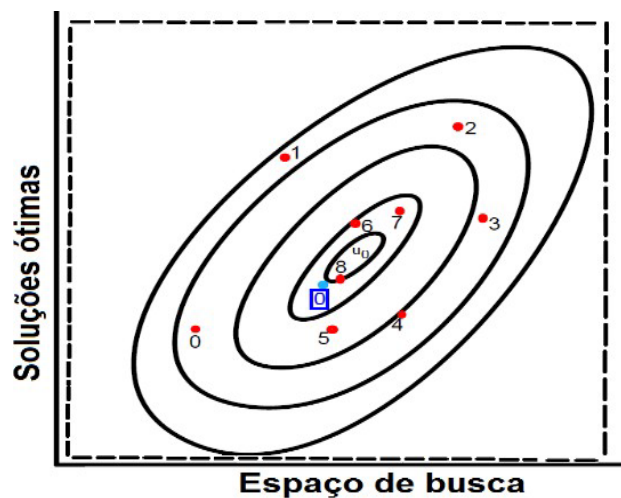
$$x_{i,j+1} = \begin{cases} u_0 & \text{if } f(u_0) \leq f(x_{i,j}) \\ x_{i,j} & \text{caso contrário} \end{cases} \quad (3.8)$$

Nessa equação:

- $i = 1, 2, \dots, M$;
- $j = 1, 2, \dots, \text{dim}$;
- $f(u_{i,j})$ é o valor de aptidão do melhor indivíduo do vetor teste;
- $f(x_{i,j})$ é o valor de aptidão do melhor indivíduo do vetor alvo.

O processo de seleção é ilustrado na Figura 13 a seguir. O melhor indivíduo é selecionado por meio do seu valor de aptidão avaliado pela função objetivo. Nesse caso, a função objetivo utiliza a minimização, com isso, o melhor indivíduo será o que apresentar menor valor de aptidão.

Figura 13 – Processo de seleção



Fonte: com base em Storn; Price, 1997.

Ao comparar $u_{i,j}$ com $x_{i,j}$, ED integra mais fortemente a recombinação e a seleção do que outras técnicas de otimização, pois as características do vetor $u_{i,j}$ são herdadas para $x_{i,j}$.

TEMA 4 – META-HEURÍSTICAS HÍBRIDAS

As duas primeiras décadas estão focadas nas meta-heurísticas padronizadas, visto que as meta-heurísticas se apresentaram mais eficientes, ágeis e de fácil implementação do que os métodos heurísticos. Entretanto,



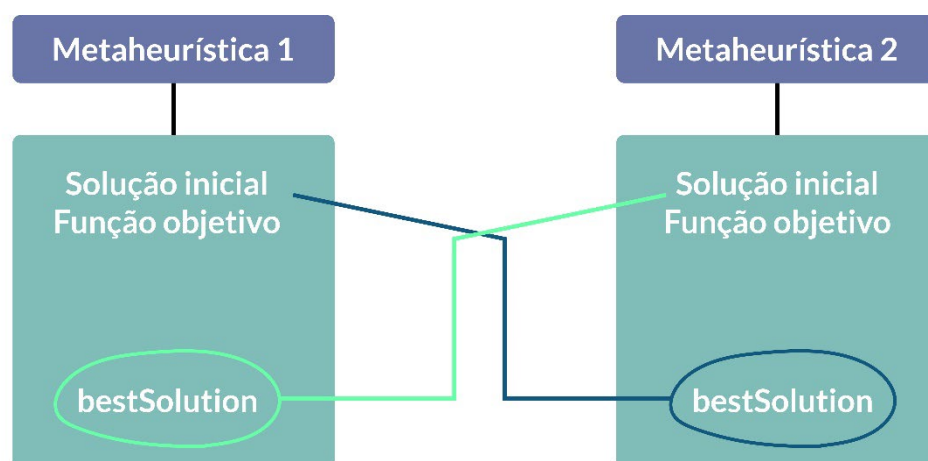
mesmo com o surgimento de novas meta-heurísticas, alguns problemas não apresentaram precisão em seus resultados, uma vez que não se existe uma única meta-heurística boa o suficiente para solucionar todos os problemas reais, ou seja, uma meta-heurística pode resultar em soluções factíveis para o problema de treinamento de redes neurais e a mesma meta-heurística pode não apresentar bons resultados para o problema da estimação de parâmetros de sistemas não lineares, por exemplo.

Dessa maneira, para tratar desse suposto problema, uma combinação habilidosa de uma meta-heurística com outra(s) meta-heurística(s), chamada de “meta-heurística híbrida”, veio como uma tentativa de suprir essa necessidade.

As meta-heurísticas híbridas pode ser mais eficientes e fornecer maior flexibilidade ao tratar com diferentes problemas do mundo real e de grande dimensionalidade (grande escala). De modo geral, as meta-heurísticas híbridas podem ser referenciadas como combinações colaborativas ou combinações integrativas.

As combinações colaborativas tratam da troca de informações entre duas ou mais meta-heurísticas sendo executadas sequencialmente. Resumidamente, pode-se dizer que essa combinação seria da seguinte forma: a meta-heurística A começa o seu processo de busca por soluções factíveis e, ao terminar todo o seu processo ela envia sua melhor solução para a meta-heurística B, e vice-versa. É uma tentativa de melhorar as soluções ou então de mantê-las, mas nunca piorar essas soluções encontradas. Na Figura 14 a seguir, exemplificamos a troca de informações entre duas meta-heurísticas.

Figura 14 – Exemplo de combinação colaborativa



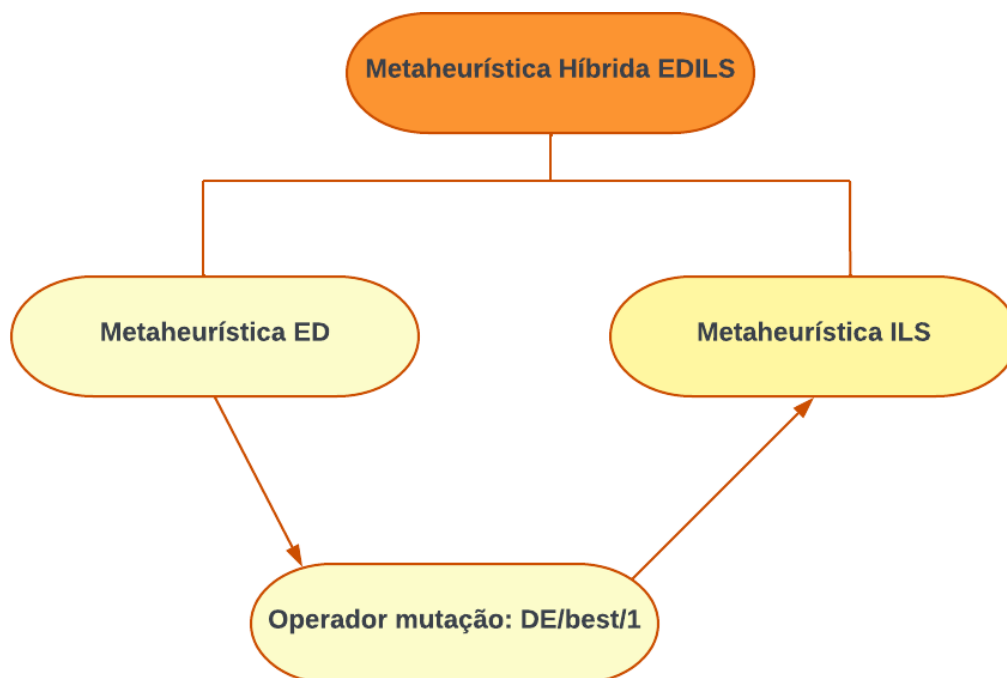
Fonte: Bergamini, 2018.



A primeira meta-heurística faz todos os seus processos e, ao terminar, guarda a *bestSolution* que foi a melhor em uma execução. Assim, *bestSolution* terá os melhores indivíduos da população da primeira meta-heurística e enviará para a segunda meta-heurística. Por sua vez, a outra meta-heurística aloca *bestSolution* na primeira fileira de sua população inicial (solução inicial) para fazer todo seu processo na tentativa de melhorar ou manter a *bestSolution* recebida. Por fim, a segunda meta-heurística guarda a nova *bestSolution*, envia para a primeira meta-heurística que é alocada também na sua população inicial. Até terminar o número máximo de execuções, essas meta-heurísticas trabalharam de modo cascata, trocando informações a cada execução.

Para as combinações integrativas, elas têm o foco de selecionar o ponto estratégico de uma meta-heurística e adicioná-lo em outra meta-heurística. Para uma compreensão melhor, pense na meta-heurística ED estudada nesta etapa. Sua principal estratégia se encontra na operação de mutação, então estaríamos colocando essa operação em outra meta-heurística. Na Figura 15 a seguir, ilustramos a junção da meta-heurística ED com a meta-heurística de Busca Local Iterativa.

Figura 15 – Exemplo de algoritmos híbridos



Fonte: Bergamini, [S.d.].

Ao combinarmos os dois métodos meta-heurísticos, temos a probabilidade de melhorar as soluções factíveis, pois são métodos de categorias



diferentes. Na literatura, muitos pesquisadores afirmam que conseguem melhorar as soluções quando se trata de problemas reais, ou seja, problemas de alta complexidade (Bergamini, 2018).

TEMA 5 – APLICABILIDADE: FUNÇÕES DE BENCHMARKS

As funções de Benchmarks são fracionadas em categorias, que são: unimodal sem restrições; multimodal sem restrições; e multimodal com restrições. Geralmente, a maior parte das funções consegue solucionar problemas Np difíceis ou de dimensões inferiores. Se maior for sua dimensão, mais será exigido do método de otimização, pois terá o aumento do esforço computacional e, por consequência, sua convergência será lenta. Cada uma das funções depende dos limites superiores (U) e inferiores (L) dos parâmetros iniciais. Na maior parte dos casos, o mínimo não depende somente da dimensão do problema, pois, ao depender do mínimo os valores da função, as dimensões selecionadas serão fornecidas para os problemas. Com isso, duas funções de testes serão apresentadas, Ackley e Rosenbrock; ambas tratam de problemas complexos.

Tabela 1 – Configuração das funções

Função de Benchmark	Dimensão	U	L	Solução ótima
Ackley	100	32	-32	$f(x) = 0$
Rosenbrock	100	10	-5	$f(x) = 0$ ou $\neq 0$

Fonte: Bergamini, [S.d.].

Ackley apresenta soluções dentro de uma região plana e central. Além disso, seu gráfico é bidimensional para ilustrar a localização de soluções ótimas. Uma possível formulação dessa função pode ser por:

$$f(x) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} (cx_i) \right) + a + \exp(1). \quad (5.1)$$

Nessa equação, $a = 20$, $b = 0,2$, $c = 2\pi$, D é a dimensão do problema e x_i é o indivíduo da população.

Quanto à função Rosenbrock, seu espaço de busca está dentro do intervalo $[-32, 32]$ e ela trata com agilidade problemas Np difíceis, que entregam



resultados precisos. Para o mínimo global, ele se localiza no vale plano, um lugar estreito e parabólico. A fórmula matemática a seguir representa a função Rosenbrock.

$$f(x) = 10 D + \sum_{i=1}^D [x_i^2 - 10 \cos(2\pi x_i)] \quad (5.2)$$

Nessa equação, D é a dimensionalidade do problema e x_i são os indivíduos da população.

Saiba mais

Vale ressaltar que temos uma infinita gama de funções de testes, tanto para a otimização mono-objetivo quanto para a otimização multiobjetivo. Para mais informações, acesse o link a seguir. Disponível em: <<https://www.sfu.ca/~ssurjano/optimization.html>>. Acesso em: 20 out. 2022.

FINALIZANDO

Chegamos ao fim de nossa jornada. Ressaltamos que este conteúdo será de suma importância para você, independentemente do cargo que exerça seja, cientista de dados, desenvolvedor(a), pois trata-se da resolução de qualquer tipo de problema real do nosso dia a dia. Iniciamos nossa jornada falando sobre problemas de otimização, expondo as duas principais formulações, o mono objetivo e multiobjetivo. Além das formulações, precisamos escolher corretamente qual método melhor se encaixa para uma resolução de precisão. Para isto, estudamos desde os métodos pioneiros (métodos heurísticos) até os mais utilizados (métodos meta-heurísticos). Estes dois métodos permitem tratar problemas de baixa até a mais alta complexidade, isto é, dependendo da dimensão do problema pode ser dito como intratável. Vimos que os métodos meta-heurísticos híbridos são uma alternativa viável para melhorar as soluções e com isto, trazem boa performance e agilidade na obtenção das soluções factíveis.

Por fim, você aprendeu como aplicar os métodos de otimização utilizando algumas funções objetivo testes em Python. De modo a entender a importância de usar as funções de testes para uma pré-análise dos métodos de otimização e assim, realizar a aplicabilidade deles para tratar qualquer tipo de problema de otimização.



REFERÊNCIAS

- ARROYO, J. E. C. **Heurísticas e meta-heurísticas para otimização combinatória multiobjetivo**. 227 f. Tese (Doutorado em Engenharia Elétrica) – Faculdade de Engenharia Elétrica, Universidade Estadual de Campinas, Campinas, 2002. Disponível em: <<http://repositorio.unicamp.br/handle/REPOSIP/260313%0Ahttp://www.bibliotecadigital.unicamp.br/document/?code=vtls000256530&fd=y>>. Acesso em: 20 out. 2022.
- BERGAMINI, M. G. **Estimação de parâmetros de sistemas não lineares utilizando um sistema colaborativo de meta-heurísticas**. 94 f. Dissertação (Mestrado em Engenharia Elétrica) – Departamento de Engenharia Elétrica, Setor de Tecnologia, Universidade Federal do Paraná, Curitiba, 2018. Disponível em: <<https://acervodigital.ufpr.br/handle/1884/56030>>. Acesso em: 20 out. 2022.
- BLANK, J. **Multi-Objective Optimization in Python**. Version 6.0, 2022. Disponível em: <<https://pymoo.org/problems/many/dtlz.html#DTLZ1>>. Acesso em: 20 out. 2022.
- CUERVAS, E.; ZALDIVAR, D.; PEREZ, M. *Advances in Metaheuristics Algorithms_ Methods and Applications*. 2018.
- KUSHIDA, J. I.; HARA, A.; TAKAHAMA, T. Rank-based differential evolution with multiple mutation strategies for large scale global optimization. 2015 IEEE Congress on Evolutionary Computation, CEC 2015 - Proceedings, , n. 1, p. 353–360, 2015.
- MAHDAVI, S.; SHIRI, M. E.; RAHNAMAYAN, S. Metaheuristics in large-scale global continues optimization: A survey. **Information Sciences**, v. 295, p. 407-428, 2015. Disponível em: <<http://dx.doi.org/10.1016/j.ins.2014.10.042>>. Acesso em: 20 out. 2022.
- NORVIG, P. The Traveling Salesperson Problem. **GitHub**, abril 16, 2018. Disponível em: <<https://github.com/norvig/pytudes/blob/main/ipynb/TSP.ipynb>>. Acesso em: 21 out. 2022.
- PRICE, K. V.; STORN, R. M.; LAMPINEN, J. A. **Differential Evolution: A Practical Approach to Global Optimization**. Germany: Springer-Verlag Berlin Heidelberg, 2005.
- STORN, R.; PRICE, K. Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. *Journal of Global Optimization*, v. 11, n. 4, p. 341–359, 1997.



ZHANG, Yudong; WANG, Shuihua; JI, Genlin. A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications. Hindawi Publishing Corporation. 2015. <https://doi.org/10.1155/2015/931256>.